



PandaBuilder

-

User Guide

The screenshot shows the PandaBuilder application window with a menu bar (File) and several configuration panels:

- CPU Options:** Includes a 'Generics' dropdown menu set to 'debug_on', a text field containing 'false', and a 'Change' button.
- Available Components:** Includes a dropdown menu set to 'input port' and an 'Add' button.
- Component Options:** This panel is divided into two columns.
 - Left column: 'Component Selection' dropdown set to 'input port' with a 'Remove' button; 'Generics' dropdown set to 'port_width' with a text field containing '8' and a 'Change' button; an 'External Component' checkbox which is unchecked.
 - Right column: 'Name' text field containing 'input port'; 'ID' text field containing 'pin'; 'Base Address (Hex)' text field containing '110'; 'Address Bits' text field containing '0'; 'Input Data' checkbox (checked); 'Interrupt' checkbox (unchecked); 'Interrupt Priority' text field containing '0'; and a 'Change' button.
- System:** Includes a 'System Name' text field containing 'NewSystem'; a 'Pack ROM into System' checkbox (unchecked); 'DataMux Style' with radio buttons for 'Synchronous' and 'Asynchronous' (selected); 'ROM Type' with radio buttons for 'Inverse clk', 'Asynchronous', 'Delayed clk' (selected), and 'External'; 'Instruction Address Width (1..10)' text field containing '10'; and 'Data Address Width (1..10)' text field containing '10'. A 'Change' button is located below these fields.

At the bottom of the window are four buttons: 'Save', 'Load', 'Generate', and 'Create SW Files'.

Logic Solutions Bründler
Winkelbühlrain 10
CH-6043 Adligenswil
Switzerland
Tel. +41 (0)76 408 83 58
E-Mail info@logicsolutions.ch
Internet www.logicsolutions.ch

Table of Contents

1 Legal Notices.....	3
2 Installation Guide.....	4
2.1 Java Runtime Environment.....	4
2.2 PandaBuilder Installation.....	4
2.3 Assembler Installation.....	4
2.4 Editor Installation.....	4
3 User Guide.....	8
3.1 PandaBuilder User Guide.....	8
3.1.1 CPU Options.....	9
3.1.2 Add Components.....	9
3.1.3 Component Options	9
3.1.3.1 Generics.....	9
3.1.3.2 Settings.....	9
3.1.4 System Options.....	10
3.2 Assembler User Guide.....	12
3.2.1 Software development Flow.....	12
3.2.2 Editor User Guide.....	12
3.2.2.1 Enable Syntax Highlighting.....	12
3.2.2.2 Assemble a Source-File.....	13
3.2.2.3 View the Listing.....	13
3.2.2.1 Creating ROM-Files.....	14
3.2.3 Assembler Syntax.....	15
3.2.3.1 Definitions and Directives.....	15
3.2.3.2 Basic Conventions.....	15
3.2.3.2.1 Comments.....	15
3.2.3.2.2 Number Formats.....	15
3.2.3.2.3 Operands for fixed Numbers.....	15
3.2.3.2.4 Segments.....	15
3.2.3.3 Declaring Peripheral Registers.....	16
3.2.3.4 Declaring Variables.....	16
3.2.3.5 Declaring Constants.....	16
3.2.3.6 Macros.....	17
3.2.3.7 Code Syntax.....	17
3.2.3.8 Interrupts.....	18
3.2.3.9 pandamac.inc File.....	18
3.2.3.10 Driver Macros.....	18
3.3 Connecting external Peripherals.....	19
3.4 Add Peripherals to PandaBuilder.....	21
3.4.1 Create Component.....	21
3.4.1.1 Create Directory.....	21
3.4.1.2 Write VHDL File.....	21
3.4.1.3 Create Register Definition File.....	22
3.4.2 Change Components.cdf File.....	23
3.4.3 Create Driver Macro File.....	24
3.4.4 Create Documentation File.....	24

Abstract

This document describes how to install and use the PandaBuilder software and the other tools for the Panda processor like the HASM assembler.

The PandaBuilder is a software which allows the user to simply click together a small system on chip with a 8-bit Panda processor. The software then automatically creates the interconnections between the different components. It also creates a top-design-file which contains the whole system. ROM can either be included in the system or externally connected. The second choice allows the user to use vendor dependent tools to make in system changes on the program code.

PandaBuilder also creates documentation files for the components and different software files to make software design easier.

It is possible to add external peripherals to the system. If some peripherals are used often, they can also be imported into PandaBuilder.

1 Legal Notices

All copyrights are reserved by Logic Solutions Bründler. If you want to use this software for commercial issues, you have to buy it.

It is not allowed to distribute the software, any design files or parts of them to third-persons.

For any sales or legal questions please write an e-mail to info@logicsolutions.ch

2 Installation Guide

This part of the document describes how to install the software you need to create panda systems and program them.

2.1 Java Runtime Environment

To run the software, JRE must be installed on your computer.

2.2 PandaBuilder Installation

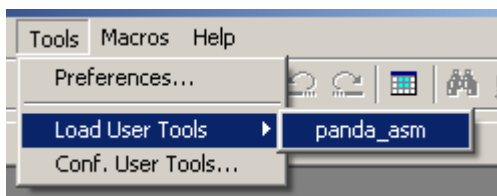
1. Copy the folder `<PandaBuilderRoot>\PANDABUILDER` to you hard-disk.
2. To start the software, run the *PandaBuilder.jar* file.

2.3 Assembler Installation

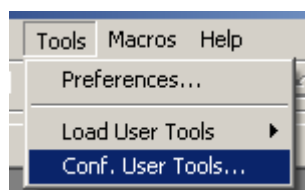
1. Copy the folder `<PandaRoot>\ASSEMBLER\HASM` to you hard-disk.

2.4 Editor Installation

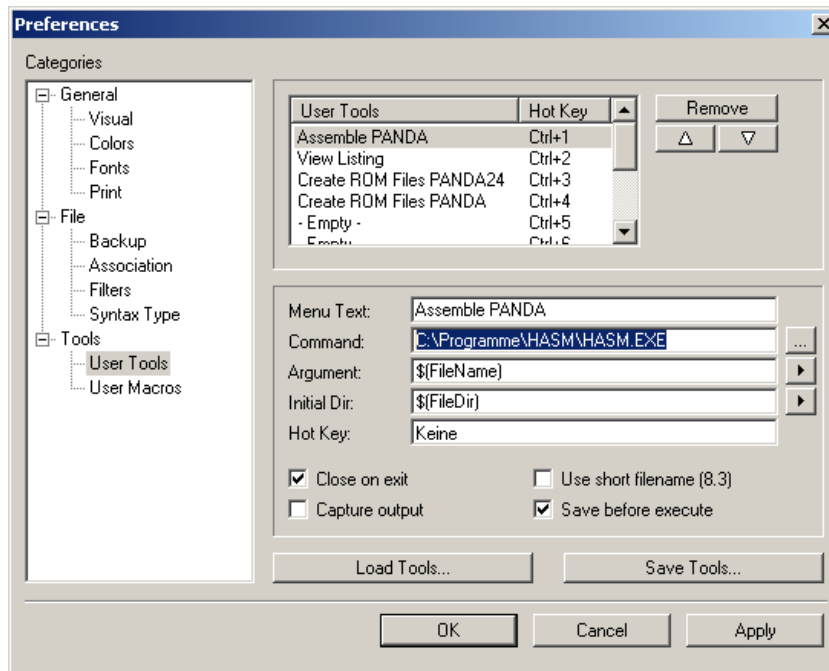
1. Execute the file `<PandaRoot>\EDITOR\cedt370r.exe`.
This will install Crimson Editor on your computer.
2. Copy the file `<PandaRoot>\EDITOR\extensions.asm` into the directory `<CrimsonEditorRoot>\link`.
3. Copy the following files to the directory `<CrimsonEditorRoot>\spec`:
`<PandaRoot>\EDITOR\panda.key`
`<PandaRoot>\EDITOR\panda.spc`
4. Copy the file `<PandaRoot>\EDITOR\panda_asm.cmd` into the directory `<CrimsonEditorRoot>\tools`.
5. Start CrimsonEditor.
6. To install the panda_asm tools, click the following menuue item.



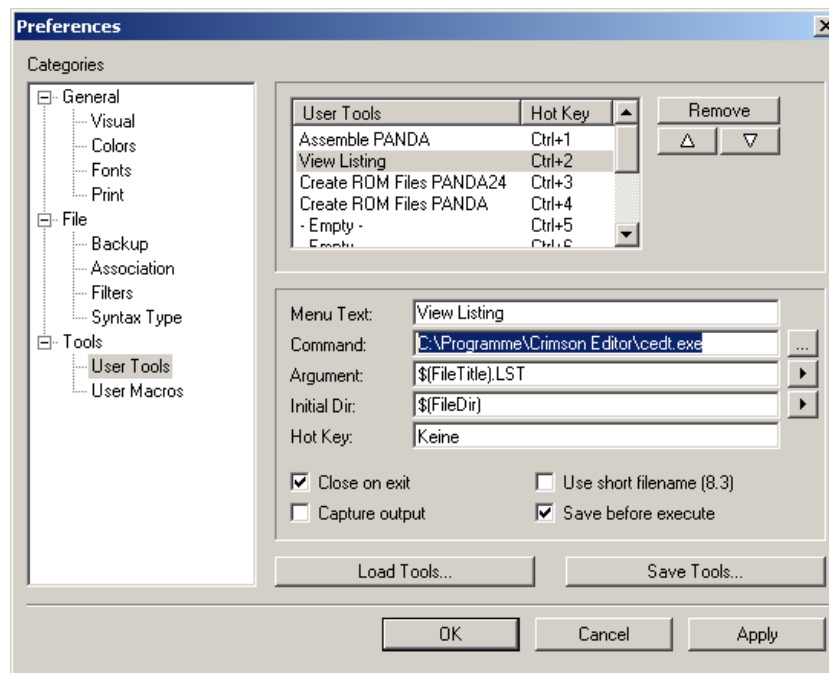
7. Now start the tools configurator.



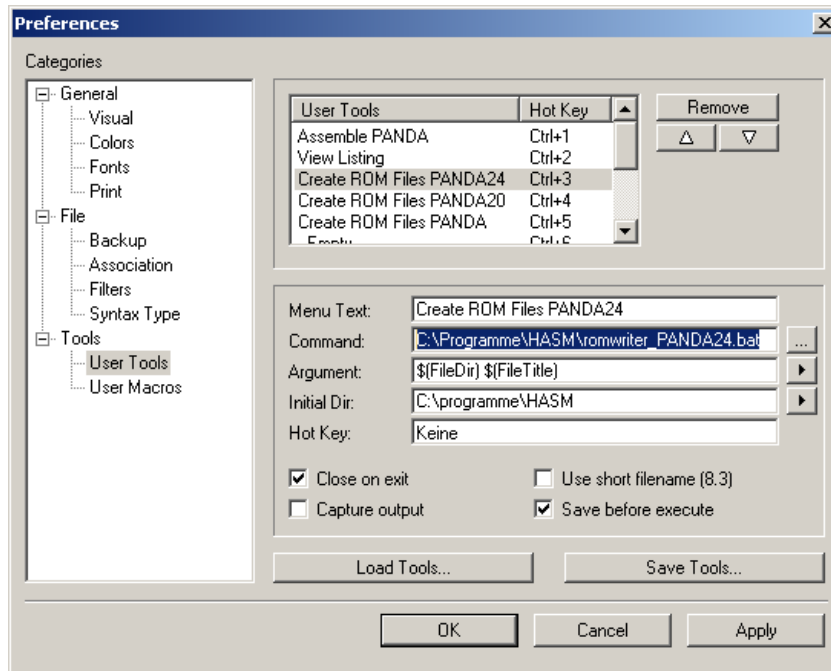
8. Set the „Command“-entry of „Assemble PANDA“ to `<HASMRoot>\HASM.EXE` from of your installation.



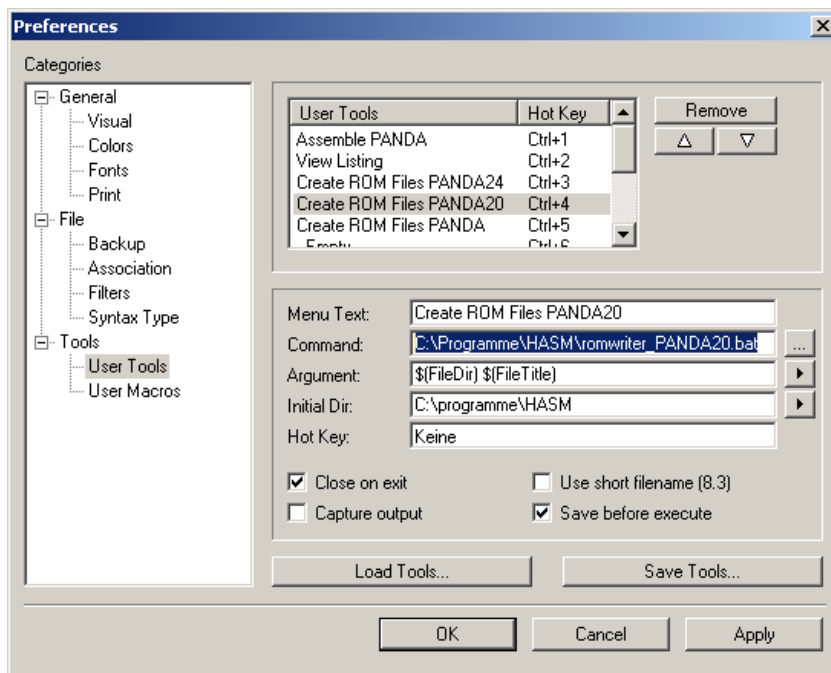
9. Set the „Command“-entry of „View Listing“ to `<CrimsonEditorRoot>\cedt.exe` from your installation.



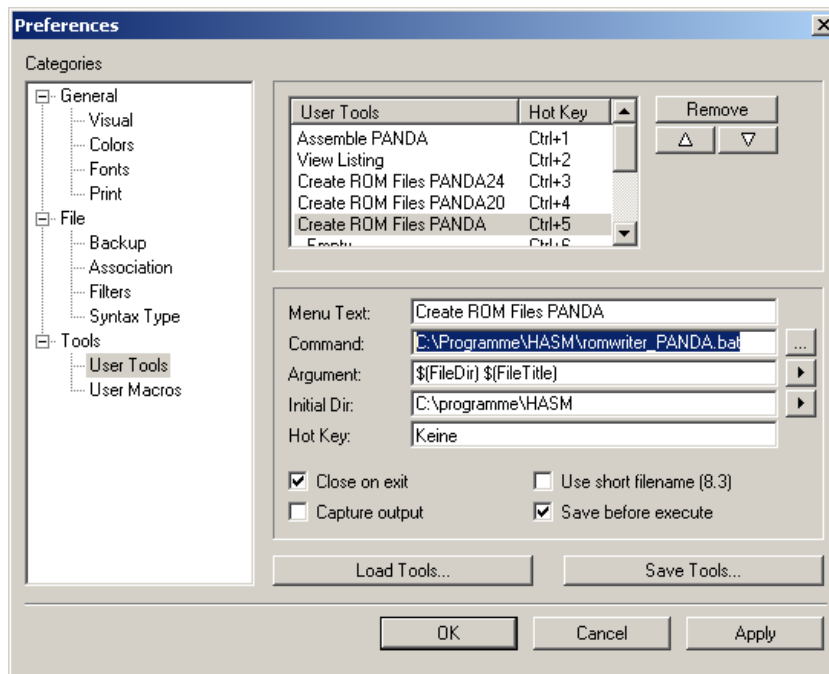
10. Set the „Command“-entry of „Create ROM Files PANDA24“ to `<HASMRoot>\romwriter_PANDA24.bat` from your installation.
Set the „Initial Dir“-entry of „Create ROM Files PANDA24“ to `<HASMRoot>`



11. Set the „Command“-entry of „Create ROM Files PANDA20“ to `<HASMRoot>\romwriter_PANDA20.bat` from your installation.
Set the „Initial Dir“-entry of „Create ROM Files PANDA20“ to `<HASMRoot>`



12. Set the „Command“-entry of „Create ROM Files PANDA“ to `<HASMRoot>\romwriter_PANDA.bat` from your installation.
Set the „Initial Dir“-entry of „Create ROM Files PANDA“ to `<HASMRoot>`



3 User Guide

This part of the document describes how to use the tools to create a panda system and program it.

3.1 PandaBuilder User Guide

The screenshot shows the PandaBuilder application window. It has a menu bar with 'File'. The main area is divided into four sections, each highlighted with a red box and a number:

- 1 CPU Options:** Contains a 'Generics' section with a dropdown menu set to 'debug_on', a text field with 'false', and a 'Change' button.
- 2 Available Components:** Contains a dropdown menu set to 'input port' and an 'Add' button.
- 3 Component Options:** Contains a 'Component Selection' dropdown set to 'input port', a 'Remove' button, and a 'Generics' section with a dropdown set to 'port_width' and a text field with '8', and a 'Change' button. To the right, there are fields for 'Name' (input port), 'ID' (pin), 'Base Address (Hex)' (110), 'Address Bits' (0), and checkboxes for 'Input Data' (checked), 'Interrupt' (unchecked), and 'Interrupt Priority' (0). A 'Change' button is at the bottom right.
- 4 System:** Contains a 'System Name' text field with 'NewSystem', a 'Pack ROM into System' checkbox (unchecked), a 'DataMux Style' section with radio buttons for 'Synchronous' and 'Asynchronous' (selected), a 'ROM Type' section with radio buttons for 'Inverse clk', 'Asynchronous', 'Delayed clk' (selected), and 'External', and two text fields for 'Instruction Address Width (1..10)' and 'Data Address Width (1..10)' both set to '10'. A 'Change' button is below these fields. At the bottom are four buttons: 'Save', 'Load', 'Generate', and 'Create SW Files'.

- | | |
|----------------------|--|
| 1. CPU Options | Here you can configure the CPU core |
| 2. Add Components | Here you can add different components to your system |
| 3. Change Components | Here you can configure the different components of your system |
| 4. System Settings | Here you can change the settings of your system |

3.1.1 CPU Options

To change a generic, select the wished generic, set the new value and press the „Change“ button.

The different generics are described in the document *PandaCPU.pdf*.

3.1.2 Add Components

To add a component, select the wished component and press the „Add“ button.

If a component with the same name already exists in the system, it will be overwritten. Therefore it is recommended to rename every component after its insertion.

You can find documentations about the different components in the directory `<PandaBuilderRoot>\components`. There is a directory for every component and a documentation file (*[componentName].pdf*) in it.

3.1.3 Component Options

In this part of the window you can change component options. Therefore you have to select the component you want to change. All components are listed with their name.

3.1.3.1 Generics

To change a generic, select the wished generic, set the new value and press the „Change“ button.

3.1.3.2 Settings

To change the settings, change their value as wished and then press the „Change“ button.

Name

Displayed name of the component, is also used in some comments of SW and HW files. Note that the name must be without whitespaces.

ID

Short name of the component. The registers of the component are named `[ID]_[register]`. External ports of the component are named `[ID]_[port]`. So choose the component ID carefully. It should make sense, be unique and short (3 to 4 characters). Not that whitespaces are not allowed.

Base Address

Base address of the component. This setting is used to define where the component can be found in the memory map.

IMPORTANT: If a component has [N] address bits, make sure that the lowest [N] bits of the base address are zero. Otherwise the system won't work!

Address Bits

Defines how many address bits are used by a component. A component with two address bits for example can have at maximum four registers, one with three address bits can have maximally eight.

Input Data

Should be selected if the CPU can read data from a component. This option is only enabled for external components. For internal components it is set automatically.

Interrupt

If a component can generate an interrupt, this setting is enabled. If it is selected, the interrupt is connected to a interrupt line of the CPU.

Interrupt Priority

If more than one interrupt is pending at one time, the interrupt with the highest interrupt priority number is served first.

IMPORTANT: Every interrupt must have a unique interrupt priority and the CPU-generic *num_of_ints* must be set to the highest interrupt priority number.

External Component

This is only a display, not a setting. It shows whether a component is internal (in the generated system file) or external (only connections to the component are generated).

3.1.4 System Options

System Name

The system name is used for the naming of the interconnect- and the system-file. The system file is named *[systemName]_system.vhd* and the interconnect file is named *[systemName]_interconnect.vhd*. It is also used as the default file name to save the system.

DataMux Style

Synchronous	Allows higher clock frequencies but the processor needs one extra clock cycle for all instructions that read data from memory or peripherals.
Asynchronous	No extra clock cycles for read instructions. Clock frequency may be reduced if a system has a lot of components.

ROM Type

Asynchronous	Connections for an asynchronous ROM are provided.
Inverse clk	Connections for a synchronous ROM are provided. The ROM clock is the inverted system clock. Therefore no additional clock cycles are needed, but clock frequency may be reduced.
Delayed clk	Connections for a synchronous ROM are provided. The system clock is used as ROM clock. Therefore one additional clock cycle is needed per instruction.

Pack ROM into System

If this option is selected, no connections for a ROM are provided. Instead, the ROM is included in the System. If the ROM Type is „Asynchronous“ the file *AsyncVHDL_[asmFilename].vhd* is used as ROM. Otherwise the file *SyncVHDL_[asmFilename].vhd* is used as ROM. Both files are located in the directory *<projectRoot>\[SystemName]SW* and can be created by the assembler (see 3.2.2.1 Creating ROM-Files)

Instruction Address Width

This value sets the width of the instruction address bus and therefore the maximal number of instructions. The value must be between 1 and 10.

Data Address Width

This value sets the width of the data address bus and therefore the maximal number of memory cells / registers that can be accessed. The value must be between 1 and 10.

Change Button

After any change, you need to press the „Change“ button. Otherwise changes are not applied.

Save

Save the system. The *<projectRoot>* directory is set to the directory the file is saved in.

Load

Load a saved system.

Generate

Generates the system. Three directories are created in the *<projectRoot>*:

<projectRoot>\[systemName]HW

All HW files (except the ROM files, if ROM is packed into system) are created in this directory.

<projectRoot>\[systemName]SW

This directory is used for software files. The „Generate“ command creates some files in it:

<i>[systemName]_memorymap.txt</i>	Memory map of the system
<i>[systemName]_instructionset.txt</i>	Instruction set of the system
<i>regdef.inc</i>	Includefile for the assembler with all definitions of the systems components
<i>[componentName].drv</i>	Includefile for the assembler with driver macros for a component

<projectRoot>\[systemName]Docs

In this directory, a datasheet is generated for every component of the system. Each datasheet is saved as *[componentID].pdf*. All names for registers and driver macros in the component datasheets are generic. To access a register use *[componentID]_[registerName]*. To use driver macros use *[componentID]_[driverMacro]*.

To access the *TMR* register of a timer with the component ID *tm1* for example, *TM1_TMR* must be used.

Create SW Files

This command creates three files in *<projectRoot>\[systemName]SW*

[systemName].asm

This is the template for the assembler file. You *can* simply fill in your software and compile it. If a file with this name already exists, no new file is generated. If you want to regenerate the file, you have to delete the existing file or rename it.

IMPORTANT: Do not change the name of this file!

pandamac.inc

This is an include file for the assembler file. It defines some pseudo instructions for the Panda processor like JNEQ or JEQ. They make programming easier.

regacc.inc

This is an include file for the assembler file. It defines some macros to set registers or set/clear bits in registers.

3.2 Assembler User Guide

3.2.1 Software development Flow

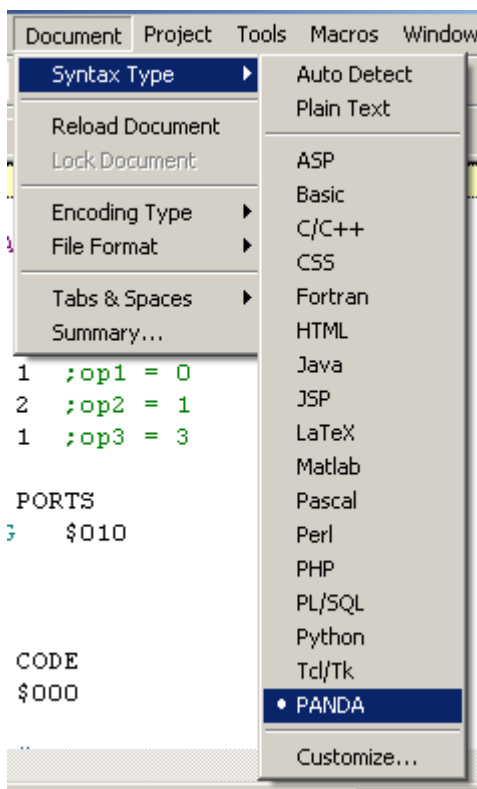
1. Write assembler source file in Crimson Editor
2. Assemble source file with HASM (tools menu in Crimson Editor)
3. View the list file and check it for errors (tools menu in Crimson Editor)
4. Create vendor dependent- and VHDL-ROM files with the romwriter tool (tools menu in Crimson Editor)
5. Program the device and test/debug the code in system

3.2.2 Editor User Guide

To follow the different steps of this chapter, please start Crimson Editor and open the file `<HASMRoot>\test.ASM` from your installation.

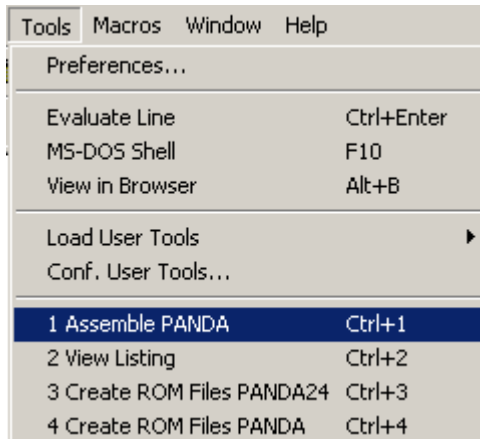
3.2.2.1 Enable Syntax Highlighting

To enable syntax highlighting click the following menu item:



3.2.2.2 Assemble a Source-File

Use the tools menu or press „Ctrl+1“ to start the assembler:

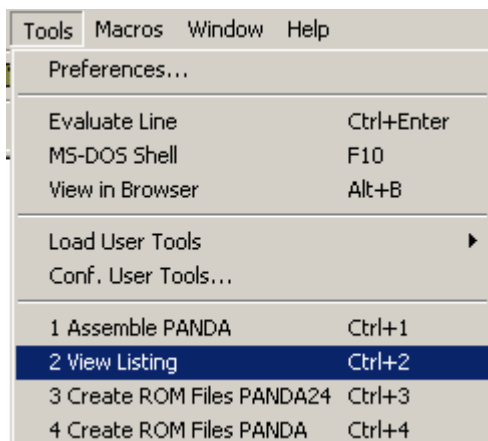


The corresponding Intel .hex File to your assembler program will now be created. You can find it under the same path as the .asm-file under *<FileName>.hex*.

The corresponding List file is also under the same path and named *<FileName>.lst*.

3.2.2.3 View the Listing

Use the tools menu or press „Ctrl+2“ to view the List-File from the assembler.



Check the list file for errors. Errors are marked above the codeline they occurred. At the end of the list file, the total number of errors is noted.

You can enable syntax highlighting again as described in 3.2.2.1 Enable Syntax Highlighting.

3.2.2.1 Creating ROM-Files

Use the tools menu or Press „Ctrl+4“/“Ctrl+5“ to create ROM files for the Panda processor. You have to use the following tools:

Create ROM Files PANDA

For Panda systems with *instruction address width* ≤ 10 and *data address width* ≤ 10 .

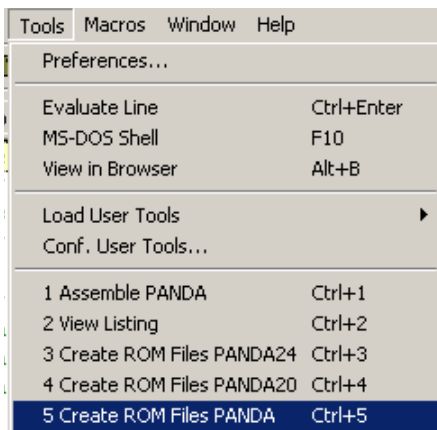
The first line of the .asm file is *#CPU PANDA*

Create ROM Files PANDA20

For Panda systems with *instruction address width* > 10 or *data address width* > 10

The first line of the .asm file is *#CPU PANDA20*

It is important to use the fitting ROM writer tool. The PANDA ROM writer creates 16-bit instructions, the PANDA20 ROM writer creates 20-bit instructions. If you use the false one, your program will not work.



The romwriter tool creates adequate ROM-initialization files in the following formats:

.mif	Altera and Xilinx Block-Rom initialization
.coe	Xilinx Block-Rom initialization
.mem	Lattice Block-Rom initialization
.vhd	Asynchronous and synchronous ROM in VHDL for simulation and vendor independent implementation

All these files are created in the same directory as the <filename>.asm file is in. They are named as follows:

```
<filename>.mif
<filename>.coe
<filename>.mem
AsyncVHDL_<filename>.vhd
SyncVHDL_<filename>.vhd
```

3.2.3 Assembler Syntax

Let's go through the file `<HASMRoot>\test.ASM` from your installation and have a look at the different parts of it and their functionality.

3.2.3.1 Definitions and Directives

Firstly the assembler needs to know, for which processor it should generate code. This is done with the following directive in the first line of the file:

```
#CPU PANDA
```

If you want to include files, you can do this with the include-directive. The following example shows how to include the File „PANDAMAC.INC“ which is in the same folder as the .asm file.

```
#INCLUDE "PANDAMAC.INC"
```

The next command tells the assembler to create a list of all used labels in the .lst file. If you don't want this label list, just leave this directive away.

```
#SYMLIST
```

3.2.3.2 Basic Conventions

3.2.3.2.1 Comments

Everything after a semicolon is interpreted as comment. If you want to write longer comments, you have to start each line of them with a semicolon.

3.2.3.2.2 Number Formats

All numbers are interpreted as decimal normally. Hexadecimal numbers must end with a „h“. Binary numbers must end with a b. All numbers must start with a digit.

```
200      ;Decimal Number
ffh      ;Is not interpreted as number because it does not start with a digit
0ffh     ;Hexadecimal Number ff = 255(Decimal)
1010b    ;Binary Number 1010 = A(Hexadecimal) = 10(Decimal)
```

3.2.3.2.3 Operands for fixed Numbers

For calculations the following operands can be used:

<code>^</code>	Bitwise Exor	<code>*</code>	Multiplication
<code>/</code>	Division	<code>%</code>	Modulo
<code>-</code>	Subtraction	<code>+</code>	Addition
<code><<</code>	Shift Left	<code>>></code>	Shift Right
<code>&</code>	Bitwise And	<code> </code>	Bitwise Or

Just an example to show the use of such operands:

```
OR      #1<<4      ;Set bit 4 in Akku
```

3.2.3.2.4 Segments

Code segments are indicated with the CSEG-directive, data segments with the DSEG-directive.

3.2.3.3 Declaring Peripheral Registers

Each peripheral should be defined as data segment at a specified address. Use the following commands for this:

```
DSEG      <PeripheralName>
REGORG    <Address>
```

It is important to use the REGORG directive and not the ORG-directive for datasegments. Otherwise the assembler will generate errors.

For every peripheral register one word of memory must be reserved. This is done through the following command:

```
<RegName> RMW      1          ;Reserve 1 Word for <RegName>
```

Its also possible to reserve more than one word. This is used for peripherals which contain memory. To do this, just replace the „1“ with the memory size.

```
<Label>    RMW      256        ;Reserve 256 Words for <Label>
```

If you want to use the same address for different register names (e.g. different input and output ports), just leave away the RMW-directive until the last used register-name.

```
<RegName1>
<RegName2> RMW      1          ;Reserve 1 Word for <RegName1> and <RegName2> together
```

It is also possible to define the address of each register as constant with the EQU-directive. This is not recommended, because the assembler does not generate errors if some addresses are used twice.

```
<RegName> EQU      <Address>
```

3.2.3.4 Declaring Variables

Variables should be defined as data segment in the RAM-addressspace. The different variables are then defined the in the same way as the peripheral registers.

```
                DSEG      VARIABLES
                REGORG    <RAM-Startaddress>
<Var1>         RMW      1          ;Reserve 1 Word for <Var1>
<VarSpace>     RMW      8          ;Reserve 8 Words for <VarSpace>
```

3.2.3.5 Declaring Constants

Constants are defined with the EQU-directive.

```
<ConstName>    EQU      <Value>
```

3.2.3.6 Macros

For small codesegments which are often used, you can define a macro. Afterwards the macro can be used as any other operator and the assembler replaces it with the defined instructions.

Macros are only used for codesements which are so small, that it makes no sense to implement them as subroutine. Bigger codesements should be implemented as subroutines, because the macro needs the full codespace everytime it is used. The subroutine only needs the codespace once and a call instruction everytime it is used.

A macro can be defined as shown in the following example:

```
<MacroName>      MACRO
                  <Instructions>
                  ENDMACRO
```

To use the macro, just write its name as an instruction and the assembler will replace it by the defined instructions.

3.2.3.7 Code Syntax

In all examples of the following chapter, the LDA instruction is used. For a complete list of all instructions please refer to the document *PandaCPU.pdf*.

Each line can begin with label (used as jump- and call-target) followed by the instruction and, depending on the instruction, an operand. A line can also only contain a label without instruction or an instruction without a label. In the second case the label must be replaced by a whitespace.

```
<label1>          ;Line with label but no instruction
<label2> LDA <operand> ;Line with label and instruction
          LDA <operand> ;Line with instruction but no label
```

The operand can be a register, a variable or an address as number for direct addressing. The usage of numbers is not recommended, because it is bad coding style.

```
LDA <RegisterName>
LDA <VariableName>
LDA 5                ;Not recommended
```

The operand is a fix number for instructions with immediate values. To indicate immediate values the route symbol is used.

```
LDA #100             ;Hex or binary values are possible too
```

Indirect addressing is indicated by an @-symbol. The indirect addressing register is changed by the number after the @-symbol after the execution of the instruction. This number can be positive or negative.

```
LDA @1               ;Load Akku with the Value at the Address specified in the
                      ;..ind. Addr. Reg., increment ind. Addr. Reg afterwards
```

When using indirect jumps, the value 0 after the at-symbol specifies a jump to the actual line. Positive values indicate forward jumps, negative values indicate backward jumps.

```
Label1 LDA #1
       JMP @2           ;Jump to Label3
Label2 LDA #2
Label3 JMP @-3          ;Jump to Label1
```

If you want some code to be located at a fixed address, use the ORG directive. Use this directive only if you really need to know the exact address of some code (e.g. Interrupt Vectors).

```
ORG 3FFh
```

3.2.3.8 Interrupts

The interrupt Vectors are located at the end of the program memory. Normally the Interrupt0 is at the address 0x3FF. If you make the instruction address bus smaller the interrupt vectors are changed too.

```
        CSEG Interrupts
        ORG 3FEh
IntVect1 JMP <InterruptServiceRoutine1> ;Interrupt Vector 1 at Address 0x3FE
IntVect0 JMP <InterruptServiceRoutine0> ;Interrupt Vector 0 at Address 0x3FF
```

3.2.3.9 pandamac.inc File

Before you use this file, you should check whether all instructions used by the *PANDAMAC.INC* file are enabled in your HW-implementation of the Panda processor. If they are not, you cannot use the corresponding macros or you have to change them.

3.2.3.10 Driver Macros

If you use one driver macro different times, it is often better to pack it into a subroutine which can be called with one instruction instead of generating all instructions each time the macro is used.

3.3 Connecting external Peripherals

If at least one external component is added to the system, different control signals for external components are added to the system. You can find a short description of them below. For more information please look at the document *PandaCPU.pdf*.

ext_cpu_data_wait

data_wait signal of the CPU. If the CPU tries to read or write data and this line is high, it is stopped until the *ext_cpu_data_wait* signal is low again. Then the read/write access is done. This line is used for all external components together.

ext_cpu_data_wre

wr signal of the CPU. This signal indicates a write access. This line is used for all external components together.

ext_cpu_data_rd

rd signal of the CPU. This signal indicates a read access. This line is used for all external components together.

ext_cpu_data_adr

data_adr signal of the CPU. This signal presents the data-address-bus. Each external component uses some bits of it to determine which register of the component is accessed.

ext_cpu_data_out

data_out signal of the CPU. This signal is the write-data-bus. It is used for all external components together.

[componentID]_cs

Chipselect signal for each external component. The component is accessed if this signal is high. A chipselect signal is generated for each external component.

[componentID]_int

Interrupt signal for external component. The component must hold the signal high until the interrupt is cleared by the CPU. This signal is only generated for components that generate interrupts.

[componentID]_dout

Data from the component to the CPU. This signal is only generated for component that can be read by the CPU.

On the next page there is an example for the connection of two external components.

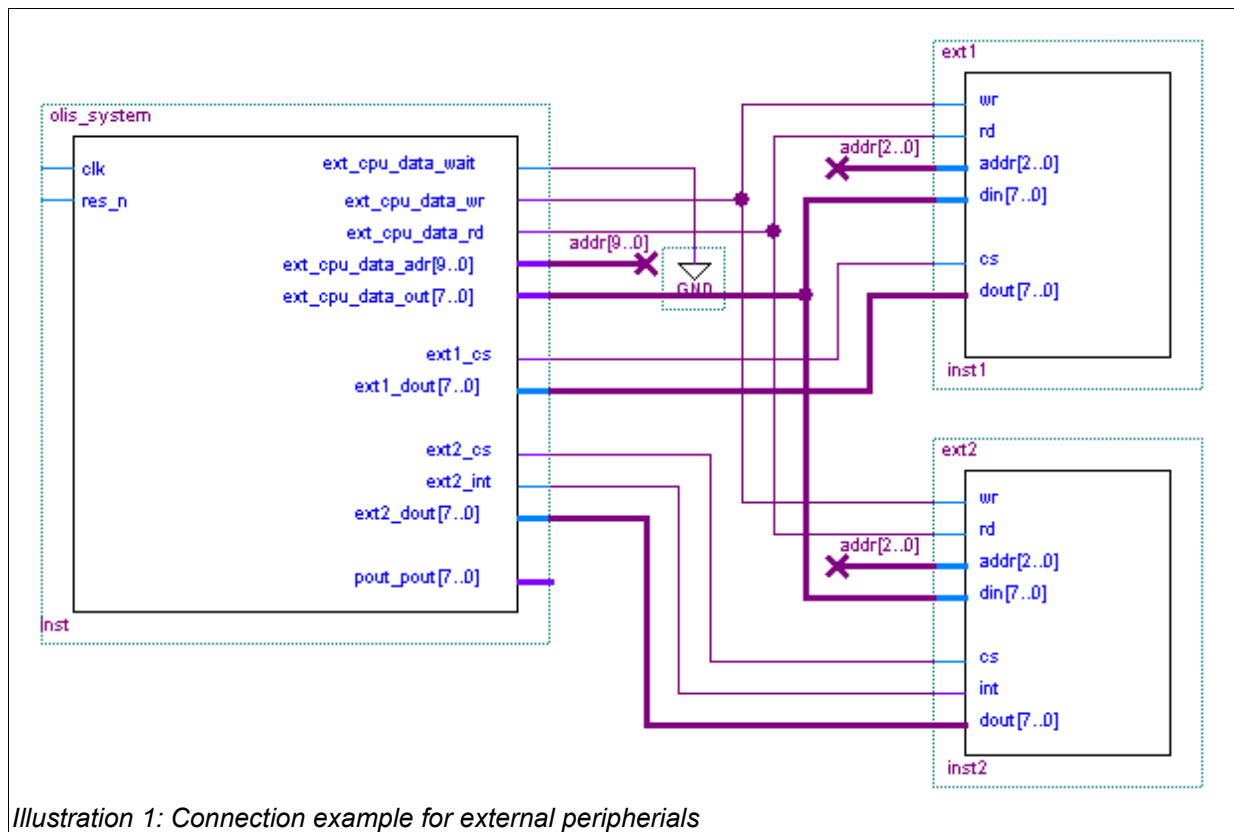


Illustration 1: Connection example for external peripherals

3.4 Add Peripherals to PandaBuilder

It is possible to import your own peripherals into the PandaBuilder software. The following chapter describes how to do this. If something should not be clearly described, have a look at existing components.

3.4.1 Create Component

3.4.1.1 Create Directory

First you have to create a directory for the new component:

`<PandaBuilderRoot>\components\[componentName]`

3.4.1.2 Write VHDL File

Write your component in VHDL. The VHDL file must be saved under:

`<PandaBuilderRoot>\components\[componentName]\[componentName].vhd`

The entity of the VHDL file must be named `[componentName]`.

The component must have following ports (names must be exactly the same!):

clk	std_logic	IN	System clock
res_n	std_logic	IN	Reset, low active
cs	std_logic	IN	Chipselect, high active
wr	std_logic	IN	Write, high active
rd	std_logic	IN	Read, high active
din	std_logic_vector(7 DOWNT0 0)	IN	Data from CPU to component

Following ports are optional and only implemented if used (names must be exactly the same!):

adr	std_logic_vector(x DOWNT0 0)	IN	Used address bits
int	std_logic	OUT	Interrupt output, high active
dout	std_logic_vector(7 DOWNT0 0)	OUT	Data from component to CPU

Additional external ports can be implemented. You are free in choosing their names.

3.4.1.3 Create Register Definition File

The register definition file can be created with any text editor. It must be saved as:

`<PandaBuilderRoot>\components\[componentName]\[componentName].rdf`

The file syntax is defined as follows:

[registerName]:[registerSize];

Registers are organized from the base address of the component. If some addresses are not used, you must specify the number of unused addresses to the next used address. *[registerSize]* is in bytes. Every register definition line must end with a semicolon.

[constantName]=[constantValue];

Constants can be described as decimal (e.g. 223), hex (e.g. 0F3h) or binary (e.g. 0101b) values. Every constant definition line must end with a semicolon.

ENDRDF;

The file must end with this line.

Example

If a component with the component ID *tmr* and the base address 0x100 has the following register definition file, the corresponding entries in the *regdef.inc* file are generated as follows:

<i>.rdf File</i>	<i>entries in regdef.inc</i>			
TMR:1;	TMR_BASE	EQU	0100h	
UNUSED:2;		REGORG	TMR_BASE	
CFG:1;	TMR_TMR	RMW	1	;at 0x100
RUN=00000001b;	TMR_UNUSED	RMW	2	
ENDRDF;	TMR_CFG	RMW	1	;at 0x103
	TMR_RUN	EQU	00000001b	
	TMR_END	EQU	0103h	

3.4.2 Change Components.cdf File

To import the component into PandaBuilder you have to change the following file which is read everytime the PandaBuilder is started:

`<PandaBuilderRoot>\components\components.cdf`

You can open and edit this file with any text editor. Note that every line must end with a semicolon.

Create the following entries for your new component (the order of the statements is important!):

COMPONENT;	
NAME=[componentName];	Default name of the component
ID=[componentID];	Default id of the component
BASEADDR=[comonentBase];	Default base address of the component (hex)
ADDRWITH=[componentAddrwith];	Default address with of the component (decimal)
INTERRUPT=[TRUE/FALSE];	TRUE if the component generates an interrupt
INPUTDATA=[TRUE/FALSE];	TRUE if the CPU can read data from the component
PATH=.\COMPONENTS\[componentFolder];	Directory you created in 3.4.1.1
ENTITY=[componentEntity];	Entity name of the component
ADDSOURCEFILES=[File1],[File2];	Additional sourcefiles for components that consist of more than one file. Different files are sparated with a colon, the number of files is open. All files must be located in the same directory as the top file of the component. If no additional Sourcefiles are needed, this line can be left away. Filenames must be with extensions (e.g. file.vhd).
GENERIC=[name:defaultValue];	If more than one generic is available, different generics are separated with a colon. If no generics are available this line can be left away. It is important that the name is exactly the same as in the VHDL file.
EXTPORTS=[name:width:direction];	If more than one external port is available, different ports are separated with a colon. Direction is IN, OUT or INOUT. It is important that the name is exactly the same as in the VHDL file. If no external ports are available, this line can be left away.
ENDCOMPONENT;	

IMPORTANT: Changes in the *components.cdf* file are not applied to components that are already added to a system. To apply changed to these components, you have to remove and reinsert them into the system after you made the changes in the *components.cdf* file.

3.4.3 Create Driver Macro File

The driver macro file is optional. If you want to create drivers, you have to create a file named:

`<PandaBuilderRoot>\components\[componentName]\[componentName].drv`

To define the macros, the normal assembler syntax is used. Every „dot“ (.) is replaced with the component ID when software files are created.

Do not use labels inside the macros. If you use the JMP instruction, use it in its indirect addressing form (e.g. JMP @2).

Example

The component for the example has the component ID *tmr* and the name *timer*.

timer.drv (in components/timer)

tmr.drv (generated in [system]SW)

```
._START    MACRO
            REG_SETB  ._CFG,  ._RUN
            ENDMACRO
```

```
TMR_START  MACRO
            REG_SETB  TMR_CFG, TMR_RUN
            ENDMACRO
```

3.4.4 Create Documentation File

If a file named *[componentName].pdf* is located in the component directory, it is copied to the *[systemName]Docs* directory and renamed with the component ID on system generation.